

Mon projet TechDay

Un guide personnel pour comprendre le système et savoir quoi demander

« Je veux consulter mes fiches et interroger un chatbot qui répond à partir de ma base. »

Le résultat visible

Dans le navigateur Une application locale avec deux onglets : Base de connaissances et Chatbot.	Dans Supabase Les fiches, la fonction du chatbot et la clé API Google gardée secrète.	Dans GitHub Les versions du code et les documents de suivi.
--	--	--

Avant chaque séance

- 1. Demander un plan avant de demander du code.
- 2. Demander quelles décisions ou questions sont importantes.
- 3. Avancer une étape, vérifier le résultat, puis seulement continuer.

La règle simple

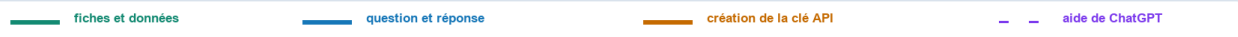
ChatGPT peut préparer, expliquer, écrire du code et tester. Je garde les décisions, les accès à mes comptes et la vérification finale.

La carte des outils

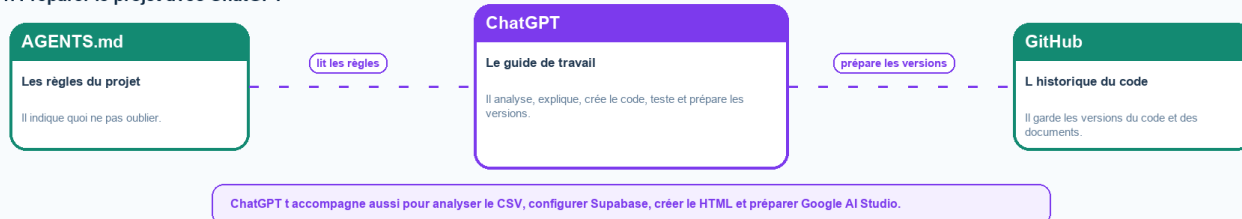
Comprendre les liaisons en un regard

La carte des outils

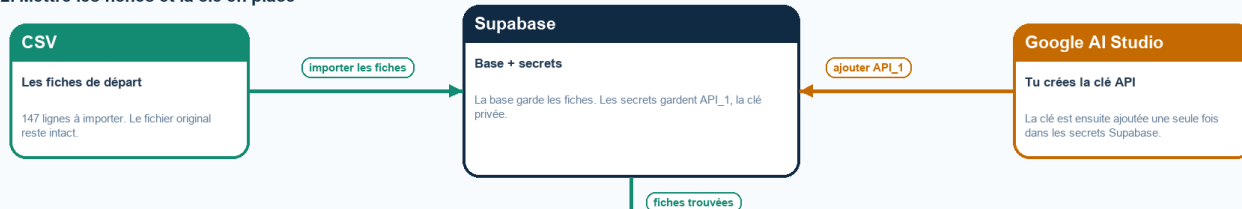
Trois moments distincts : préparer, ajouter les données, puis utiliser l'application.



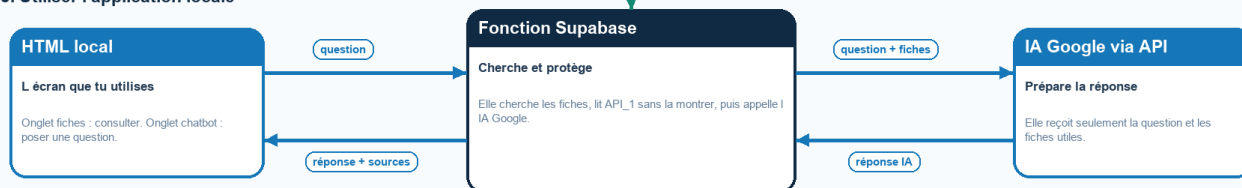
1. Préparer le projet avec ChatGPT



2. Mettre les fiches et la clé en place



3. Utiliser l'application locale



La clé ne va jamais dans le HTML ni dans GitHub : elle va de Google AI Studio vers les secrets Supabase.

ChatGPT aide à construire ce parcours ; lorsque tu poses une question, le parcours réel est HTML → Supabase → IA Google → Supabase → HTML.

Avant d'utiliser l'application : ChatGPT t'aide à lire les règles, analyser le CSV, préparer Supabase, créer le HTML et enregistrer une version dans GitHub.

Pour la clé : tu la crées dans Google AI Studio et tu l'ajoutes dans les secrets Supabase. Elle ne va jamais dans le HTML.

Quand tu poses une question : le HTML appelle la fonction Supabase. Elle cherche les fiches, utilise la clé API privée pour appeler l'IA Google, puis renvoie la réponse et les sources au HTML.

Travailler avec ChatGPT

Ce qui peut être délégué et ce que je garde en main

ChatGPT peut	Je dois
• analyser le CSV • proposer un plan • créer le HTML et les fonctions • expliquer les mots techniques • tester les résultats • préparer les commits GitHub	• définir le but • choisir ce qui est public ou privé • autoriser les connexions et les envois • saisir mes mots de passe • vérifier que le résultat me convient

La demande qui évite de se perdre

« Avant de coder, résume ce que tu as compris, propose les étapes, puis pose-moi seulement les questions qui empêchent d'avancer. Explique chaque mot technique simplement. »

Astuce

Conserver cette demande dans AGENTS.md ou dans ton guide. Elle transforme ChatGPT en guide de projet au lieu de lui demander une solution trop vite.

Le parcours en six étapes

À chaque étape : je demande, j obtiens, je vérifie

1. Définir le but

Je demande
« Crée un plan et pose-moi les questions importantes avant de coder. »

J obtiens
Un objectif clair et les décisions à prendre.

Je vérifie
Je comprends le but en une phrase.

2. Lire le CSV

Je demande
« Analyse le CSV sans le modifier. Compte les lignes et les champs manquants. »

J obtiens
Un inventaire des données à importer.

Je vérifie
Le CSV original reste intact.

3. Organiser Supabase

Je demande
« Crée ou vérifie la table de fiches. Ne supprime aucune donnée. »

J obtiens
Une table prête à recevoir les fiches.

Je vérifie
Les 147 fiches sont présentes.

4. Créer le HTML

Je demande
« Crée une page avec Base de connaissances et Chatbot. »

J obtiens
Une application locale simple à ouvrir.

Je vérifie
Les fiches s affichent et se filtrent.

5. Connecter l IA

Je demande
« Crée une clé API dans Google AI Studio, puis mets-la dans les secrets Supabase, jamais dans le HTML. »

J obtiens
Un chatbot qui utilise la fonction Supabase.

Je vérifie
Une question retourne une réponse et une source.

6. Tester et garder une version

Je demande
« Teste le parcours puis crée un commit clair et envoie-le sur GitHub. »

J obtiens
Une version vérifiée dans l historique.

Je vérifie
GitHub contient le dernier commit.

Comprendre le chatbot RAG

Une bibliothèque, pas une réponse inventée

Imagine une bibliothèque. Tu poses une question au bibliothécaire. Il cherche des fiches utiles, les donne à l'IA Google par l'API, puis elle rédige une réponse avec les références trouvées.

Le chemin actuel

1. Question Je demande par exemple : « Que sais-tu sur Blender ? »	2. Recherche Supabase cherche les mots présents dans les fiches.	3. Lecture L'IA Google reçoit au plus 12 fiches trouvées.	4. Réponse Le chatbot répond et affiche les fiches utilisées.
--	--	---	---

Ce qui fonctionne aujourd'hui

- Le chatbot répond à partir des fiches trouvées et cite leurs titres.
- La clé API Google reste dans Supabase. Les URL des fiches ne sont pas envoyées à l'IA Google.

Ce qui viendra ensuite : pgvector

Le RAG actuel cherche surtout les mêmes mots. Avec pgvector, il cherchera aussi le sens. Une question sur « animation 3D » pourra trouver une fiche sur Blender, même si le mot Blender n'est pas écrit dans la question.

Rendre l'outil plus intelligent

Les améliorations qui comptent vraiment

1. Améliorer les fiches

Donner un titre, une courte description et des étiquettes cohérentes.

2. Ajouter pgvector

Chercher les fiches par le sens, en plus des mots exacts.

3. Enrichir avec un agent

Compléter une fiche seulement avec une source vérifiable.

Les outils à utiliser

Pour améliorer le RAG

Je demande

« Ajoute pgvector. Crée des embeddings pour le titre, le texte et les étiquettes. Combine recherche par mots et recherche par le sens. »

J obtiens

Une recherche plus tolérante aux synonymes et aux formulations naturelles.

Je vérifie

Une question formulée autrement retrouve une fiche pertinente.

Pour compléter les fiches

Je demande

« Propose un agent qui enrichit une fiche uniquement avec des sources vérifiables et enregistre la source. »

J obtiens

Des fiches plus complètes, sans contenu inventé.

Je vérifie

Chaque ajout indique une source consultable.

Règle de sécurité

Une clé API Google est un accès privé. Elle reste dans les secrets Supabase. GitHub reçoit le code, pas les mots de passe ni les clés secrètes.

Modèles de demandes

Des phrases prêtes à copier pour chaque séance

Demander un plan

« Voici mon objectif. Avant de coder, propose un plan court avec les résultats à vérifier après chaque étape. »

Faire poser les bonnes questions

« Avant de commencer, pose-moi les questions qui sont indispensables. Si tu peux vérifier une information dans les fichiers du projet, fais-le sans me le demander. »

Demander une explication simple

« Explique-moi cette étape comme à un débutant : le but, ce que je dois faire, ce que je dois voir à la fin et pourquoi. »

Demander un test

« Vérifie le résultat avec un exemple concret, puis explique-moi simplement ce qui fonctionne et ce qui reste à faire. »

Demander une version GitHub

« Vérifie l'état Git, crée un commit avec un message clair et envoie-le sur GitHub sans force push. Dis-moi exactement ce qui a été envoyé. »

Mon réflexe

Je commence par le but, je demande le plan, je vérifie un résultat à la fois et je garde une version dans GitHub lorsque tout fonctionne.